AWS

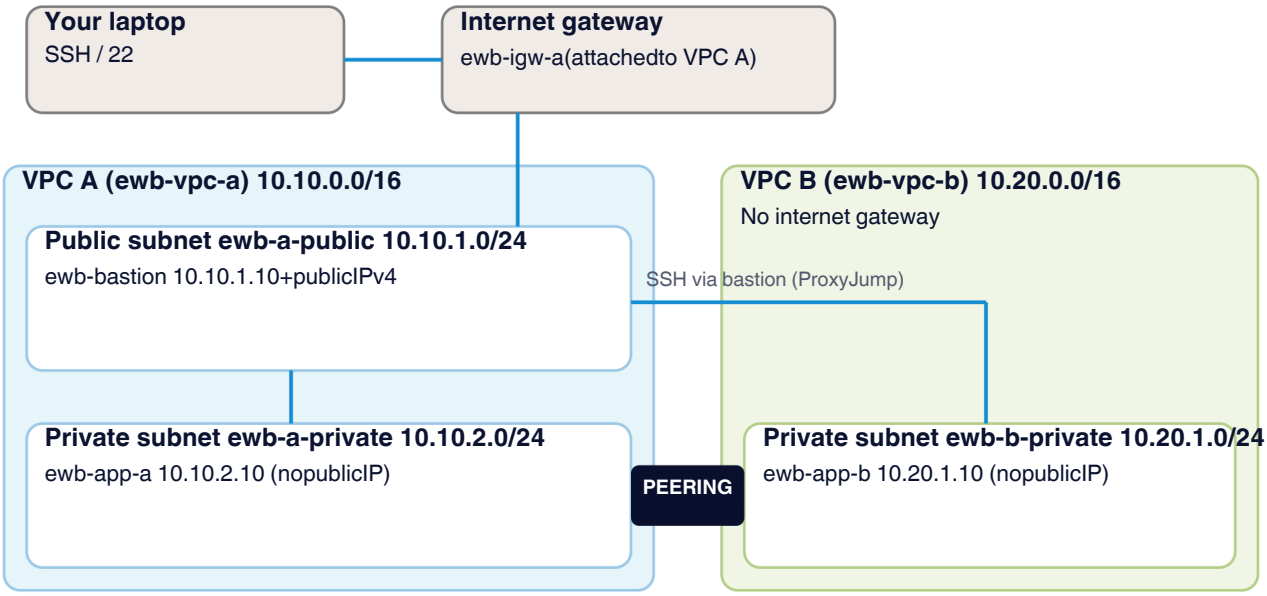
VPC Peering Project Report

Connect two VPCs. Prove private connectivity.

Student	Arjit Kumar Pandey
Batch	AWS-Internship Batch-1
Course	EWB Courses - AWS Architect Associate (Project 01)
Project tag	Project = EWB-VPC-Peering, Student = Arjit kumar pandey
AWS Region	Asia Pacific (Sydney) / ap-southeast-2, Availability Zone ap-southeast-2a
Completion date	23 October 2026

1. Architecture and inventory

TwoVPCs with non-overlapping CIDRs (10.10.0.0/16 and 10.20.0.0/16) are joined by one VPC peering connection. Only the bastion in VPC A has a public IPv4 address. App-A and App-B stay private and talk to each other over private IPs through the peering connection.



Resource inventory (actual IDs from my account)

Resource	Name / ID	CIDR / IP
VPC A	ewb-vpc-a / vpc-07f14bb0675ab2d8a	10.10.0.0/16
VPC B	ewb-vpc-b / vpc-0caffcd82d94cb4b4	10.20.0.0/16
Subnets	ewb-a-public, ewb-a-private (subnet-031f01a5f29887464), ewb-b-private	10.10.1.0/24, 10.10.2.0/24, 10.20.1.0/24
Internet gateway	ewb-igw-a (attached to ewb-vpc-a)	-
Route tables	ewb-rt-a-public, ewb-rt-a-private (rtb-064182b30be255682), ewb-rt-b-private	-
Peering connection	ewb-a-b-peer / pcx-0b40275e45e3598c6	Requester VPC A, accepter VPC B
ewb-bastion	i-068277aae07fdaf8c	10.10.1.10 + public IPv4 3.25.57.116
ewb-app-a	i-09e6a43548be15a5e	10.10.2.10 (no public IP)
ewb-app-b	i-05d2a9bd6f12bfff7	10.20.1.10 (no public IP)
Security groups	ewb-sg-bastion (sg-0bacf62f5ff78d323), ewb-sg-app-a (sg-09ee8389613da53a5), ewb-sg-app-b (sg-02862f44599691506)	-
Key pair	ewb-lab (kept only on my laptop)	-

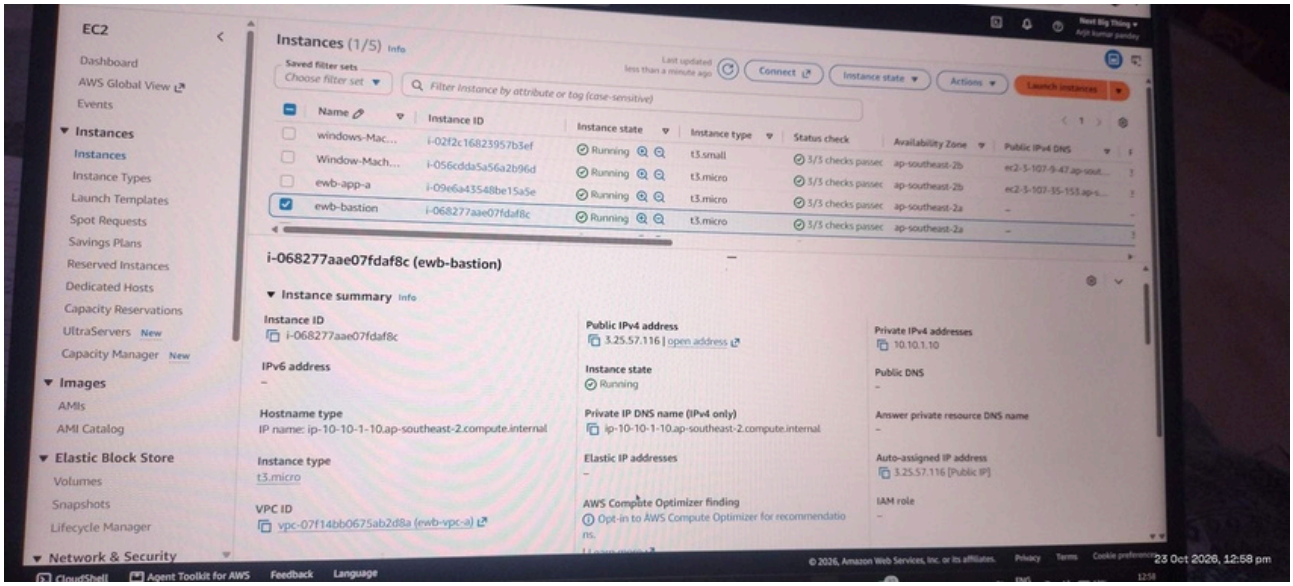


Figure 1 - Host: ewb-bastion. Instance running, 3/3 checks passed, public IPv4 3.25.57.116, private IPv4 10.10.1.10, in ewb-vpc-a.

SCREENSHOT PENDING: Peering connection ewb-a-b-peer showing Active status, requester VPC A and accepter VPC B (Checkpoint 03).

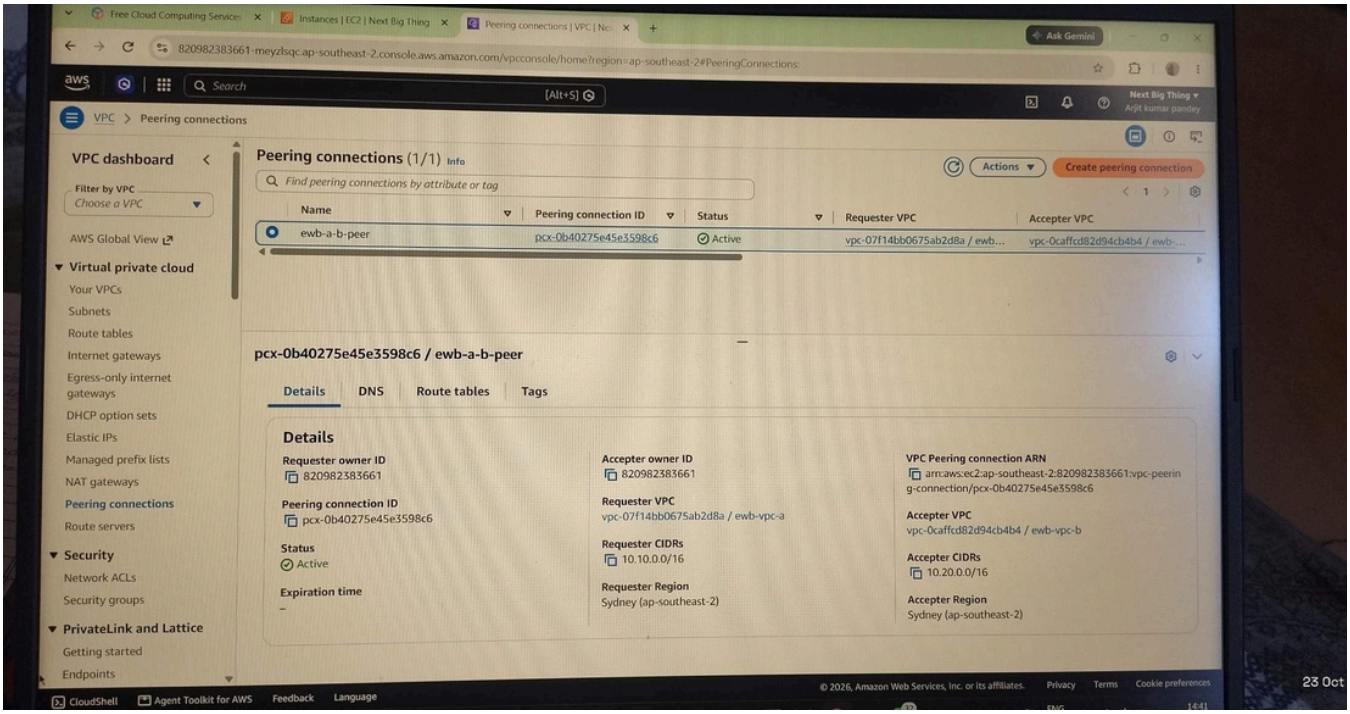


Figure 1b - Checkpoint 03. Host: AWS console. Peering connection ewb-a-b-peer (pcx-0b40275e45e3598c6) is Active. Requester: ewb-vpc-a (vpc-07f14bb0675ab2d8a, 10.10.0.0/16). Accepter: ewb-vpc-b (vpc-0caffcd82d94cb4b4, 10.20.0.0/16). Same account and Region (ap-southeast-2).

2. Subnets, route tables and security groups

Route tables (final configuration)

Route table / associated subnet	Destination	Target	Purpose
ewb-rt-a-public / ewb-a-public	10.10.0.0/16 0.0.0.0/0 10.20.0.0/16 10.10.0.0/16	local ewb-igw-a pcx-0b40275e45e3598c6 local	Bastion to App-A Bastion internet access Bastion SSH to App-B Existing VPC A route
ewb-rt-a-private / ewb-a-private	10.20.0.0/16 10.20.0.0/16	pcx-0b40275e45e3598c6 local	App-A to VPC B Existing VPC B route
ewb-rt-b-private / ewb-b-private	10.10.0.0/16	pcx-0b40275e45e3598c6	Return path to VPC A

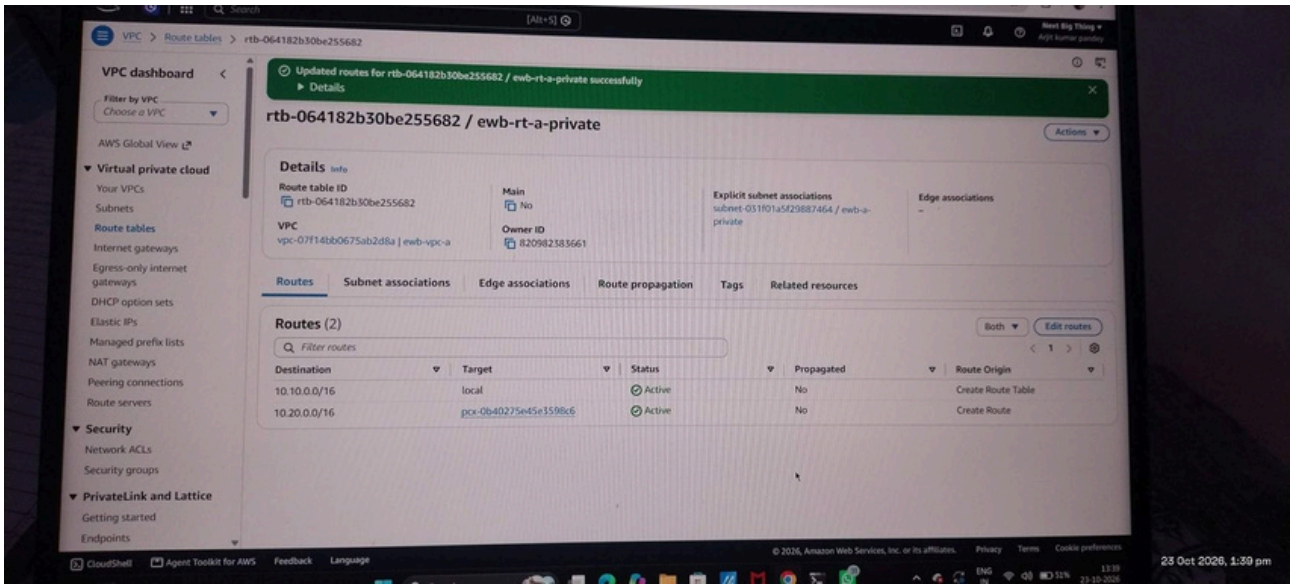


Figure 2 - Route table ewb-rt-a-private (rtb-064182b30be255682), VPC ewb-vpc-a, explicit association with ewb-a-private. Routes: 10.10.0.0/16 local and 10.20.0.0/16 -> pcx-0b40275e45e3598c6, both Active.

SCCheRcEkEpoNiSntH 0O4T s cPrEeNenDsIhNoGts: (Rroutte sta abnleds s aunbdn estu absnseot caiastsioncisa toiof nesw)b: -sret-e

Security group inbound rules

Security group (VPC)	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	M15y2 p.5u9b.l6ic2 .s3o9u/r3c2e (amdyd rpeusbsl ic/3 s2o (usr
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	Custom ICMP IPv4 - Echo Request	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	Custom ICMP IPv4 - Echo Request	10.10.2.10/32 (App-A)

Checkpoint 04 evidence: route tables and subnet associations

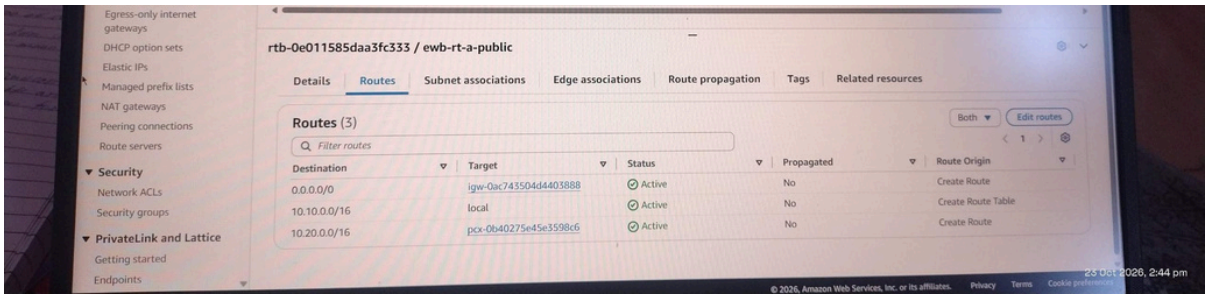


Figure 2b - Checkpoint 04. Host: AWS console. Route table ewb-rt-a-public (rtb-0e011585daa3fc333) routes: 0.0.0.0/0 -> igw-0ac743504d4403888, 10.10.0.0/16 local, 10.20.0.0/16 -> pcx-0b40275e45e3598c6, all Active.

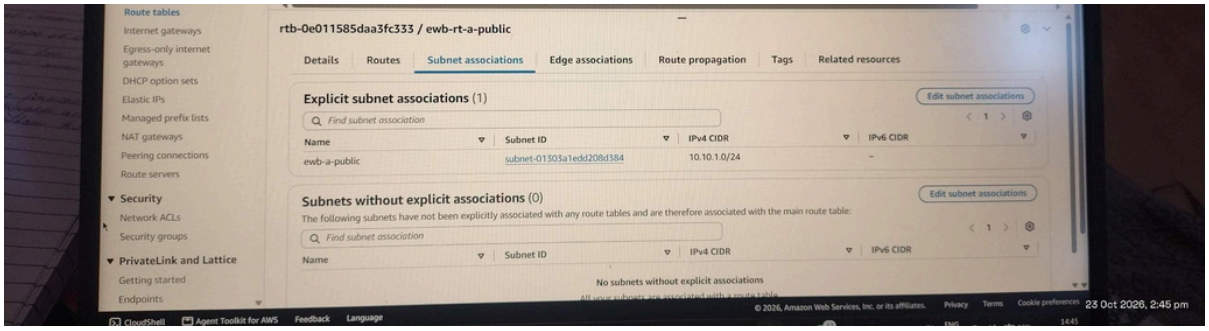


Figure 2c - Checkpoint 04. Host: AWS console. ewb-rt-a-public is explicitly associated with ewb-a-public (subnet-01303a1edd208d384, 10.10.1.0/24). No subnet uses the main route table.

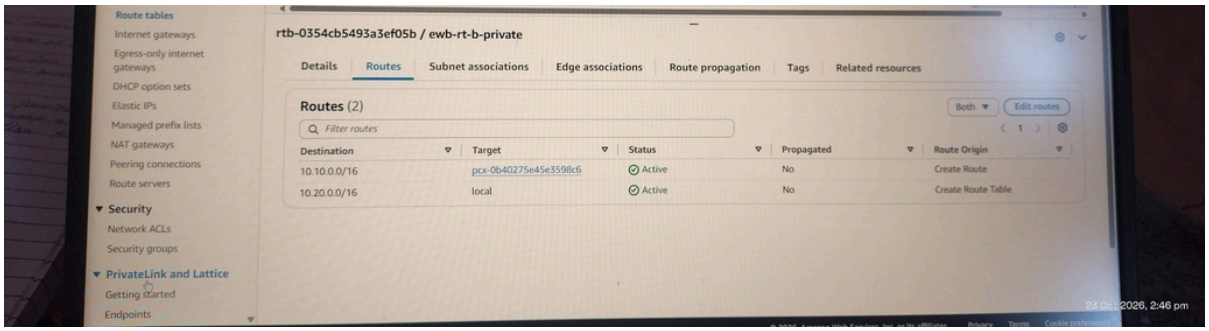


Figure 2d - Checkpoint 04. Host: AWS console. Route table ewb-rt-b-private (rtb-0354cb5493a3ef05b) routes: 10.20.0.0/16 local, 10.10.0.0/16 -> pcx-0b40275e45e3598c6 (return path to VPC A), both Active.

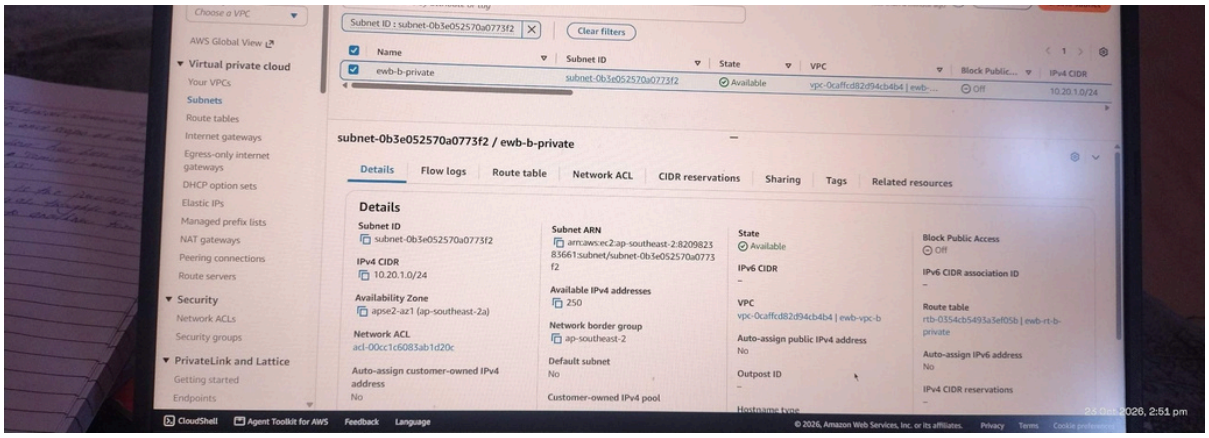
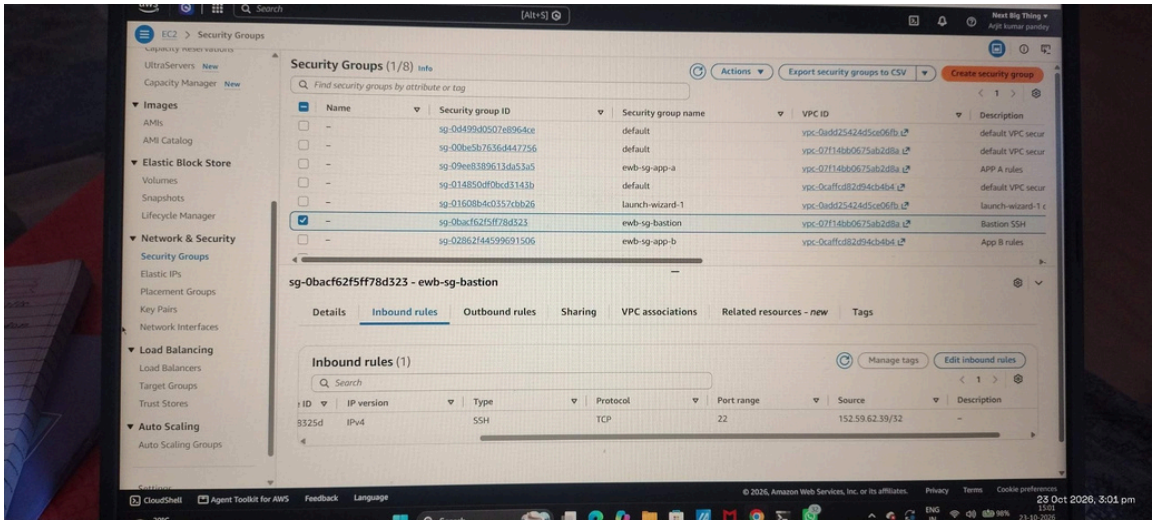


Figure 2e - Checkpoint 04. Host: AWS console. Subnet ewb-b-private (subnet-0b3e052570a0773f2, 10.20.1.0/24) in ewb-vpc-b uses route table ewb-rt-b-private (rtb-0354cb5493a3ef05b). Auto-assign public IPv4 is No.

Security group inbound rules (evidence with Source column)



FFiigguurree 33 a- -S Hecousrti:t yA WgroSu cpo enwsobl-es.g e-awpbp-s-bg -(bsags-0ti2o8n6(2sgf4-405b9a9cf66921f55f0f768),d i3n2b3o)u, S15S2H.5 T9C.6P2 .2329,/ 3a2ft e(mr tyh ep ucbolnict rsoolluerdc-efa ailudrder etesss)t.. Green banner confirms the rules were saved.

SCREENSHOT PENDING: Inbound rules with the Source column visible for ewb-sg-bastion, ewb-sg-app-a and ewb-sg-app-b

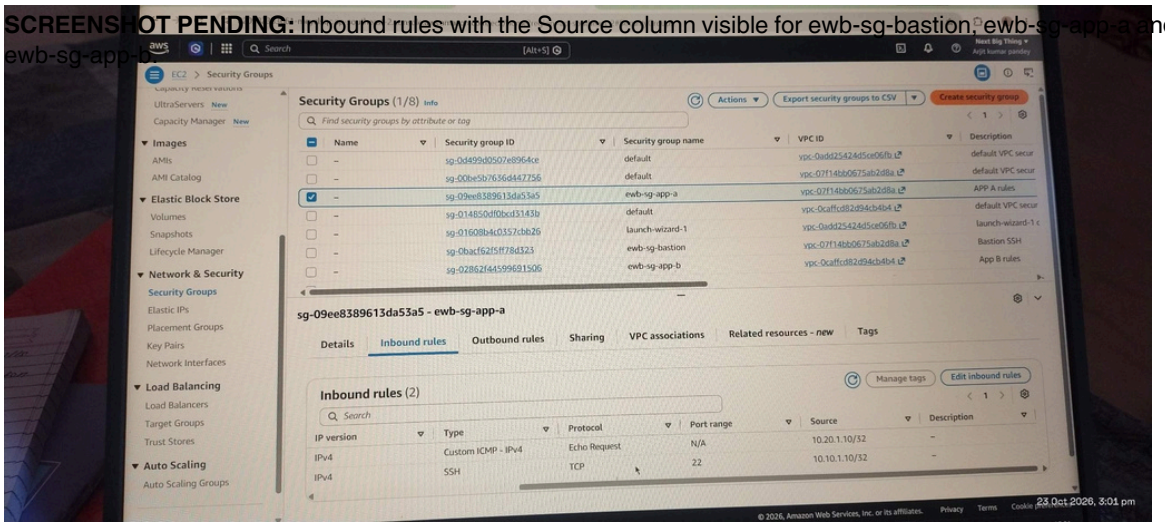


Figure 3b - Host: AWS console. ewb-sg-app-a (sg-09ee8389613da53a5), VPC A. Inbound rules (2): Custom ICMP IPv4 Echo Request from 10.20.1.10/32 (App-B) and SSH / TCP 22 from 10.10.1.10/32 (bastion private IP).

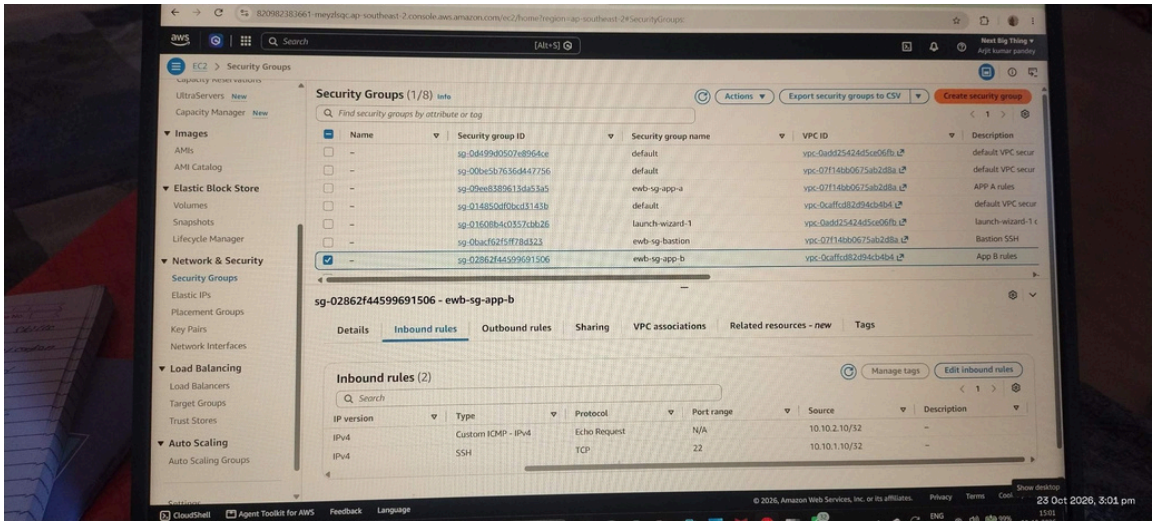


Figure 3c - Host: AWS console. ewb-sg-app-b (sg-02862f44599691506), VPC B. Inbound rules (2): Custom ICMP IPv4 Echo Request from 10.10.2.10/32 (App-A) and SSH / TCP 22 from 10.10.1.10/32 (bastion private IP).

3. Private SSH access (ProxyJump through the bastion)

The private keystayson my laptop (permissions restricted withicacls). The~/.ssh/configfileuses ProxyJump ewb-bastion, so App-A and App-B are reached through the bastion without copying any key onto an EC2 instance.

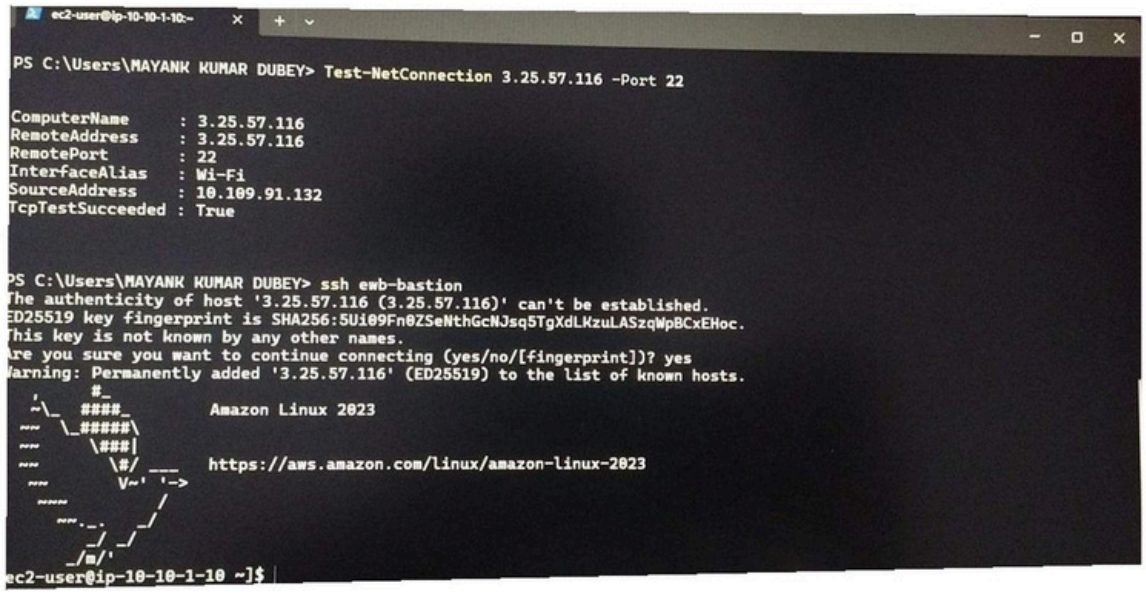


Figure 4 - Host: laptop -> ewb-bastion. ssh ewb-bastion succeeded; prompt ec2-user@ip-10-10-1-10.

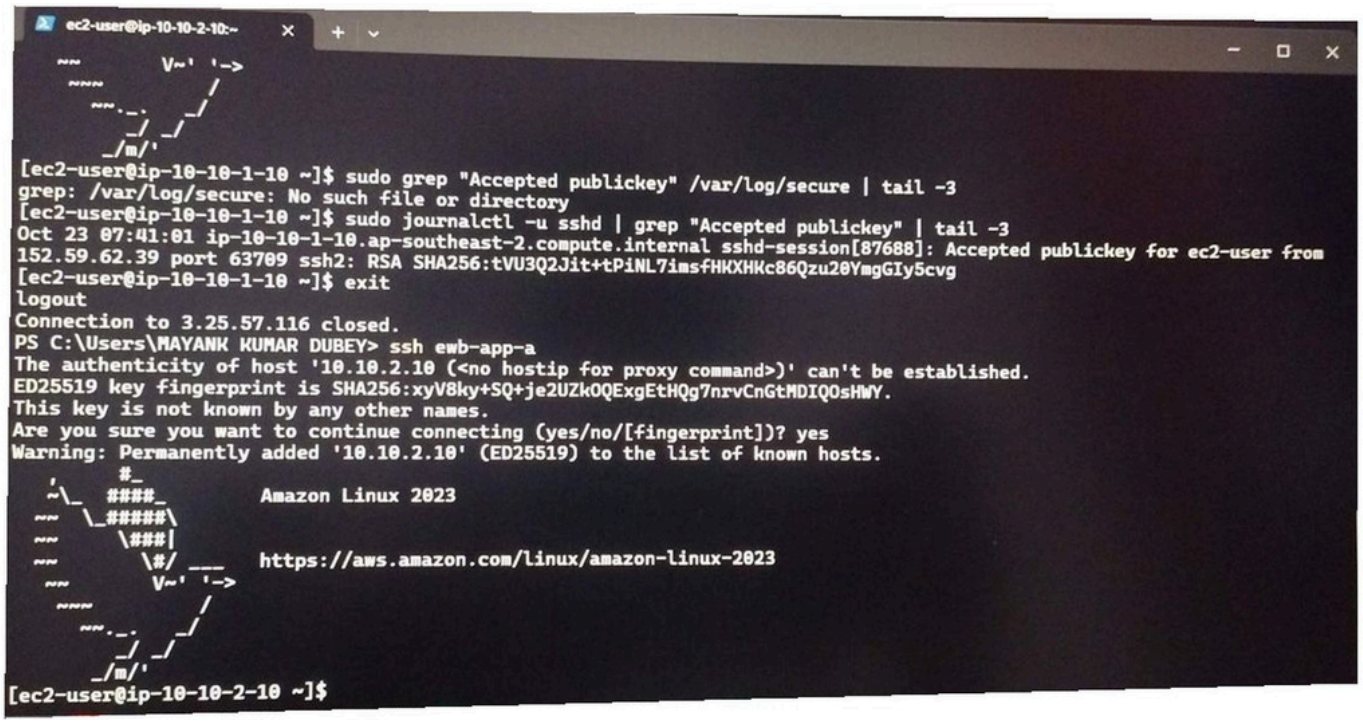


Figure 5 - Host: laptop -> ewb-app-a through the bastion (10.10.2.10). Prompt ec2-user@ip-10-10-2-10.

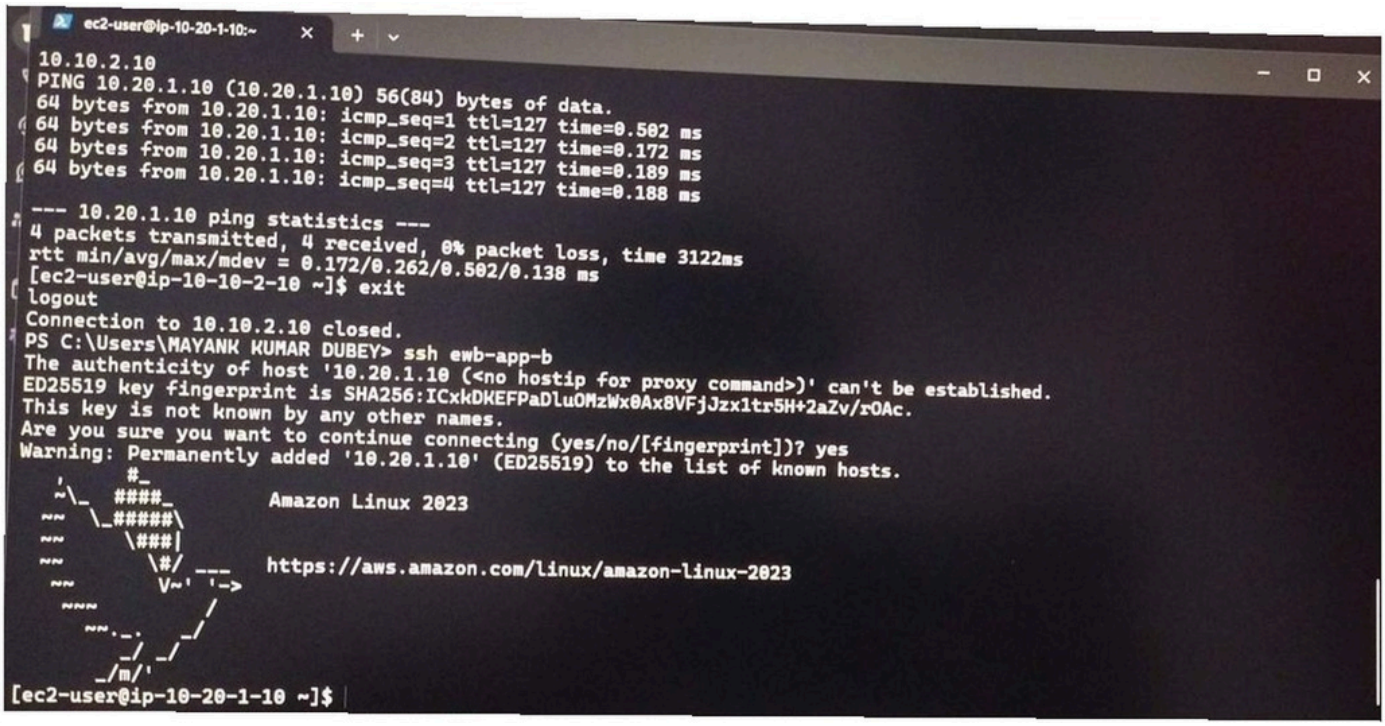


Figure 6 - Test 04 Private SSH: ssh ewb-app-b reached App-B (10.20.1.10) through the bastion. This works only because the public route table has a route to 10.20.0.0/16 via the peering connection and VPC B has the return route.

4. Connectivity tests

Test	Where / command	Result
01 Baseline (peer route removed)	ewb-app-a: ping -c 4 10.20.1.10	100% packet loss
02 Forward path	ewb-app-a: ping -c 4 10.20.1.10	4 received, 0% loss
03 Reverse path	ewb-app-b: ping -c 4 10.10.2.10	4 received, 0% loss
04 Private SSH	laptop: ssh ewb-app-b	Shell on App-B through bastion
05 Isolation	EC2 instance details of App-A and App-B	No public IPv4 on either host
06 Controlled failure	Remove ICMP Echo rule on ewb-sg-app-b, ping from App-A, then restore	100% loss, then 0% loss

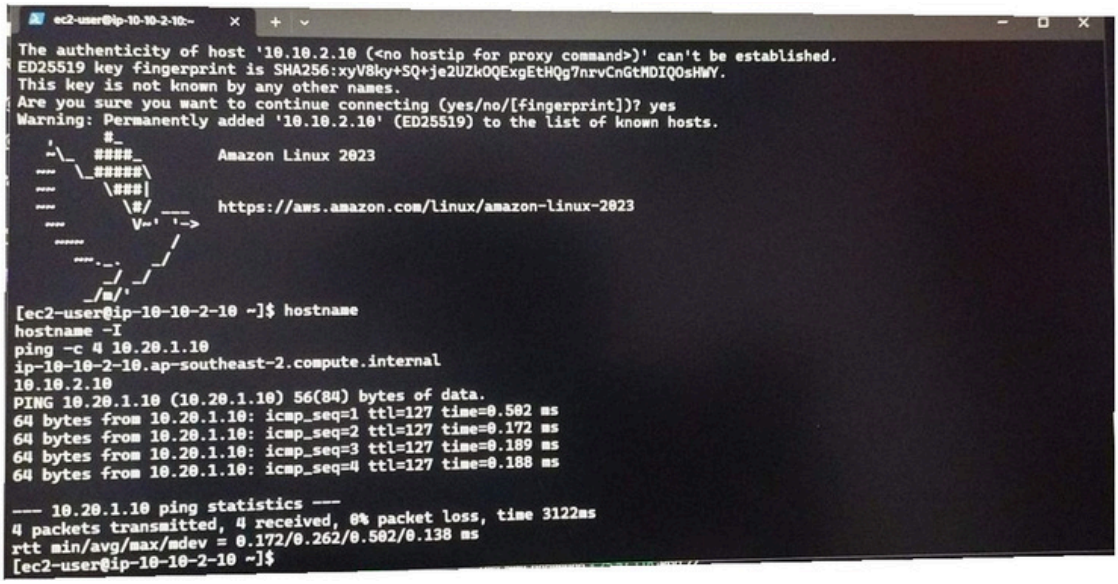


Figure 7 - Test 02 Forward path. Host: ewb-app-a (hostname -I = 10.10.2.10). ping -c 4 10.20.1.10: 4 transmitted, 4 received, 0% packet loss.

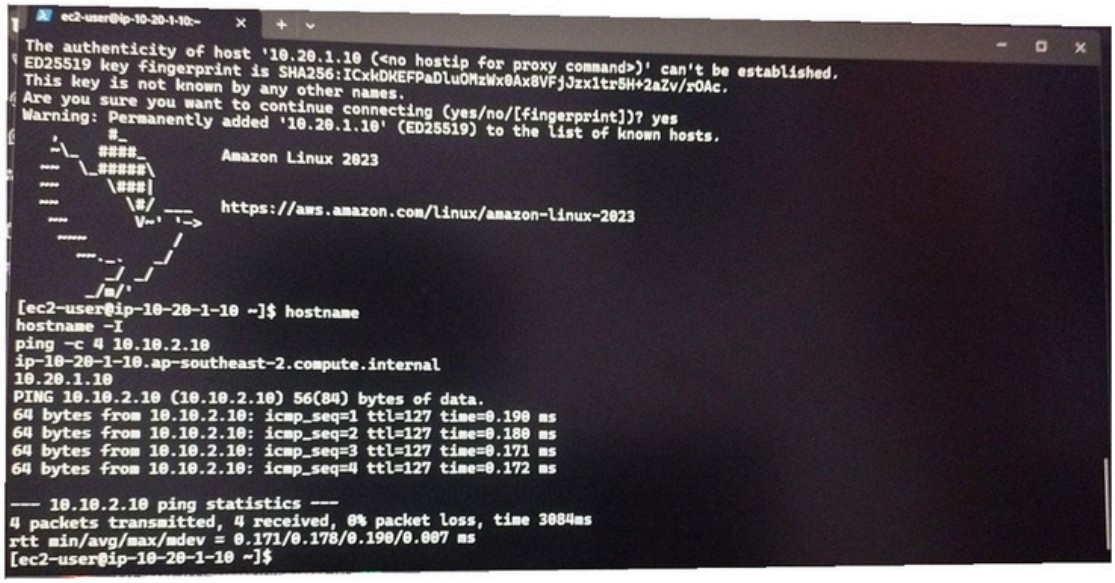


Figure 8 - Test 03 Reverse path. Host: ewb-app-b (hostname -I = 10.20.1.10). ping -c 4 10.10.2.10: 4 transmitted, 4 received, 0% packet loss.

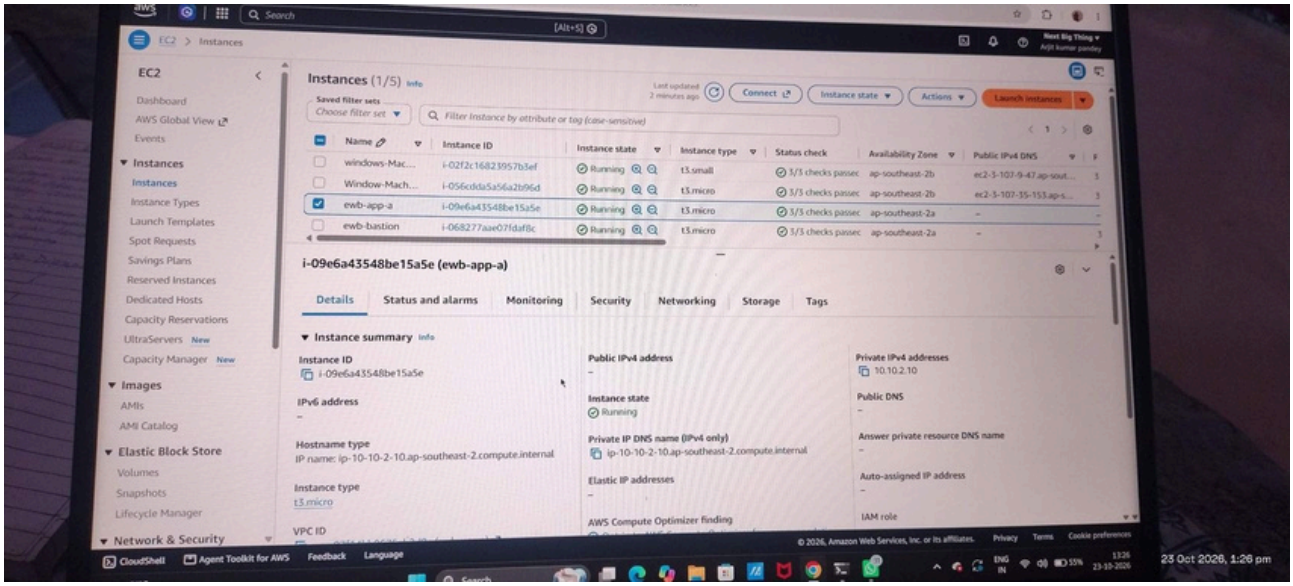


Figure 9 - Test 05 Isolation. ewb-app-a: Public IPv4 address empty, private IPv4 10.10.2.10, no Elastic IP.

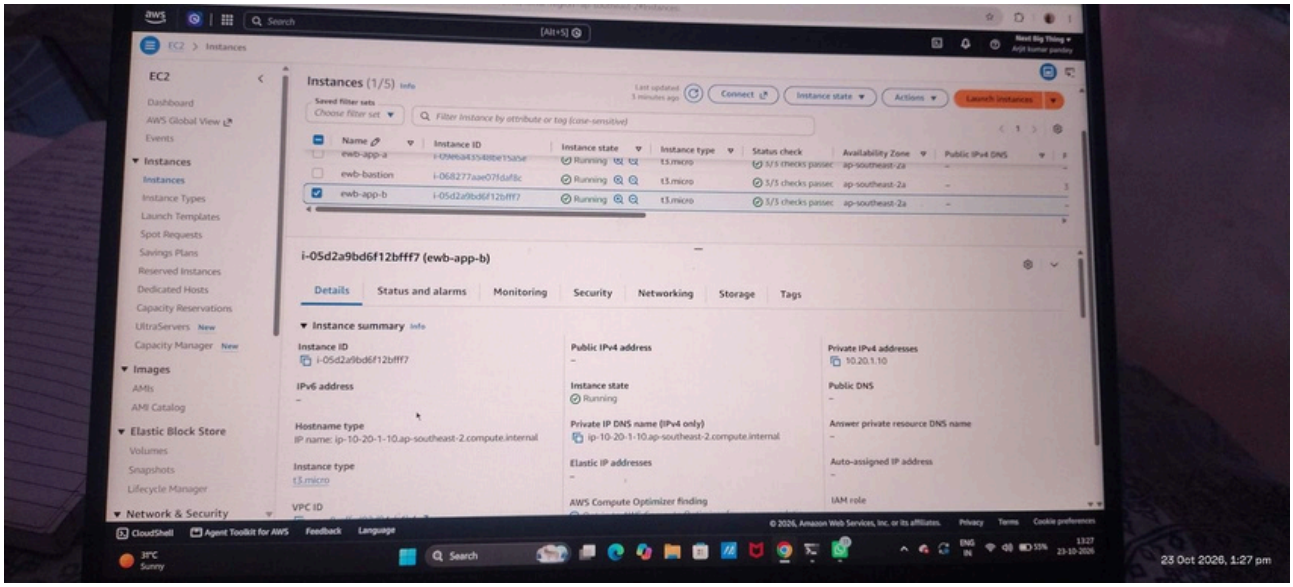


Figure 10 - Test 05 Isolation. ewb-app-b: Public IPv4 address empty, private IPv4 10.20.1.10, no Elastic IP.

Baseline failure and recovery (route test)

To show a failed test, I removed the route 10.20.0.0/16 -> pcx-0b40275e45e3598c6 from ewb-rt-a-private. App-A then had no path to VPC B, even though the peering connection was still Active. After I added the route back, the ping worked again.

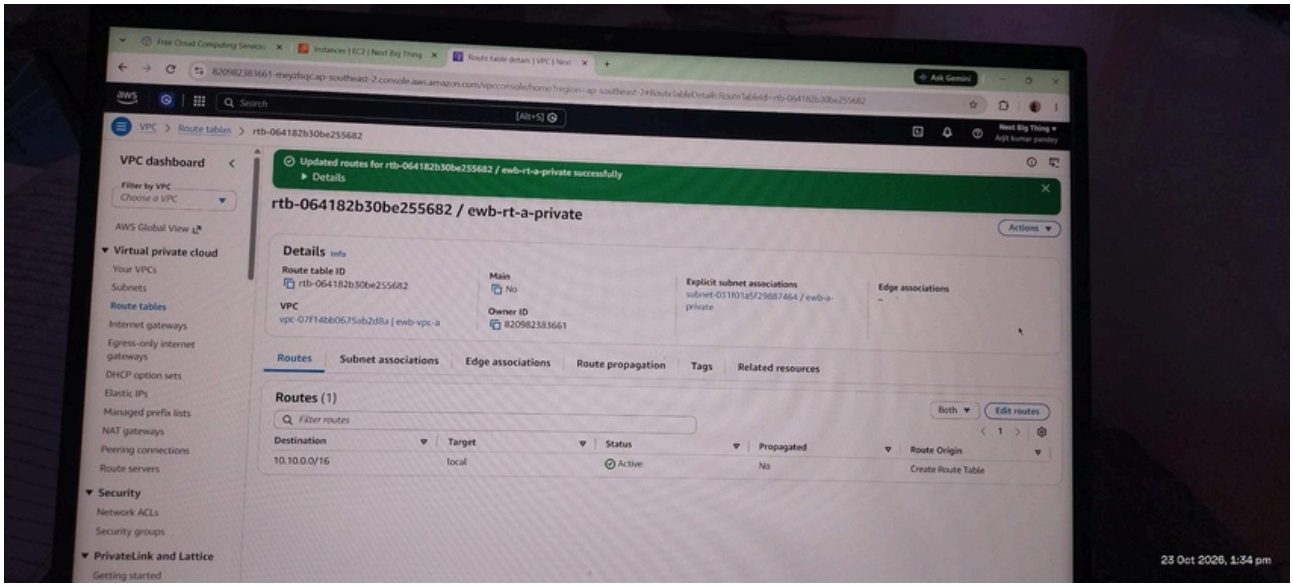


Figure 11 - ewb-rt-a-private after removing the peering route: Routes (1), only 10.10.0.0/16 local.

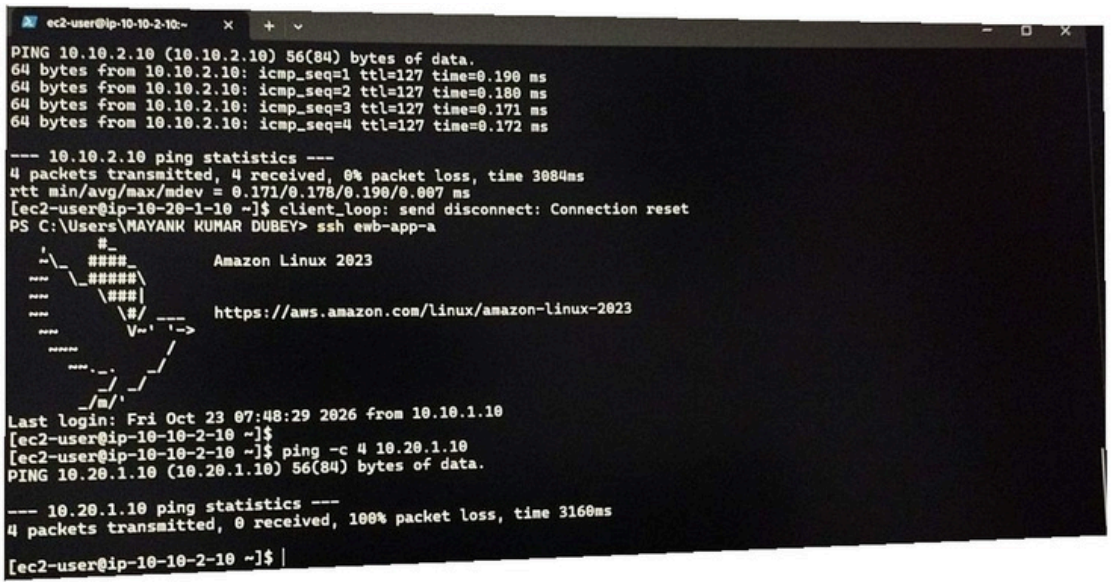


Figure 12 - Test 01 Baseline failure. Host: ewb-app-a. ping -c 4 10.20.1.10 with the route removed: 0 received, 100% packet loss.

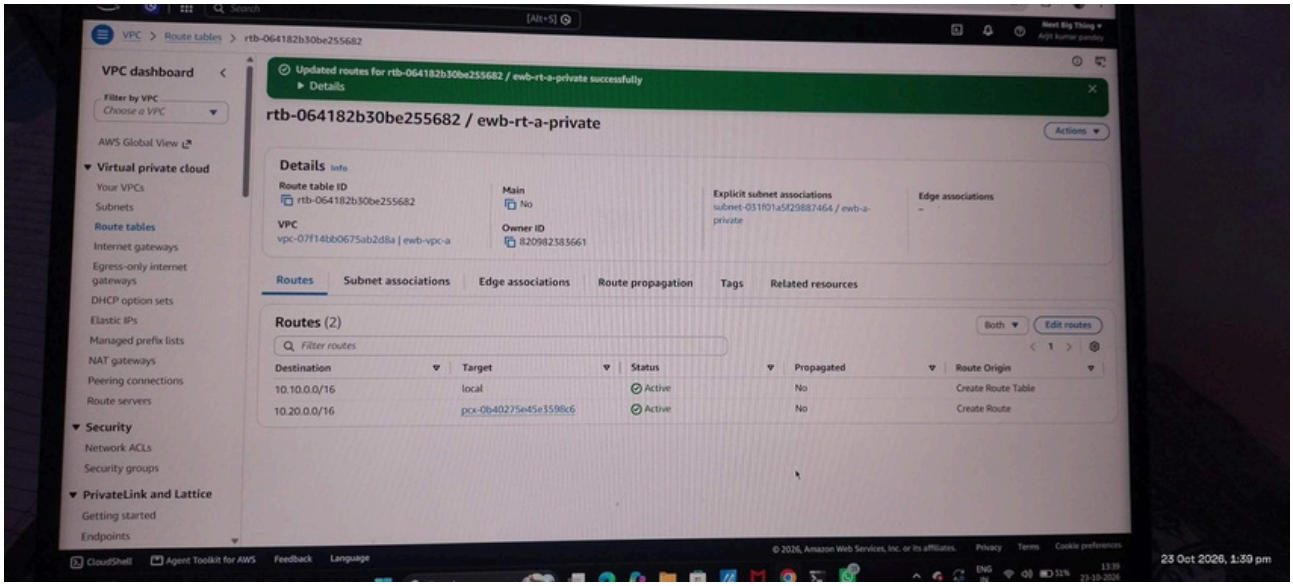


Figure 13 - Route restored: ewb-rt-a-private has 10.20.0.0/16 -> pcx-0b40275e45e3598c6 again (Active).

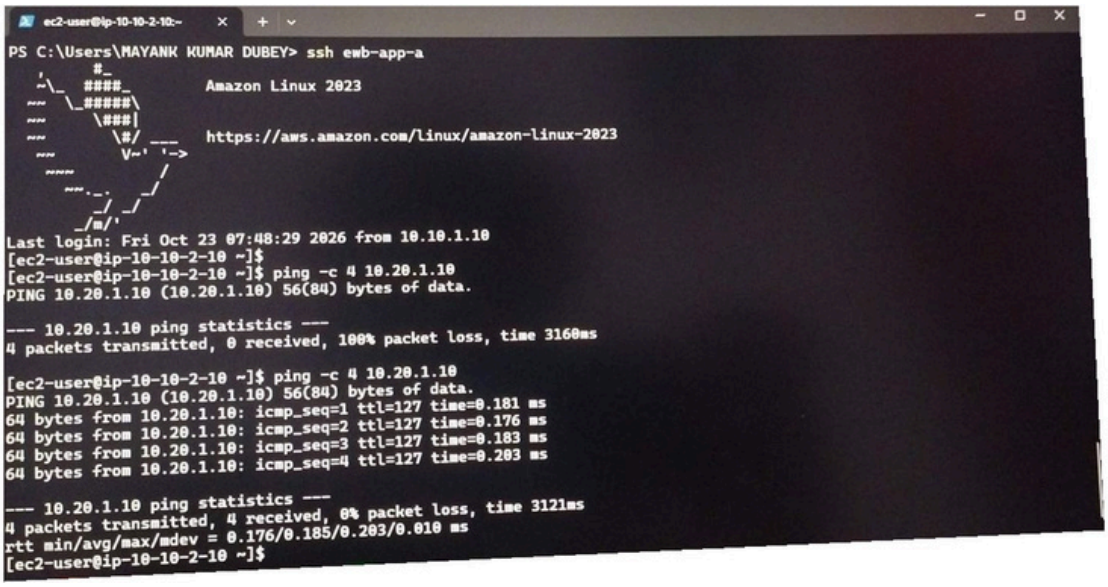


Figure 14 - Host: ewb-app-a. Same terminal: 100% loss without the route, then 4 received and 0% loss after the route was restored.

Controlled failure (ICMP security group rule)

I removed only the ICMP Echo Request rule (source 10.10.2.10/32) from ewb-sg-app-b and kept the SSH rule. App-A could no longer ping App-B because the destination security group dropped the echo request. I then restored the exact same rule and the replies resumed.

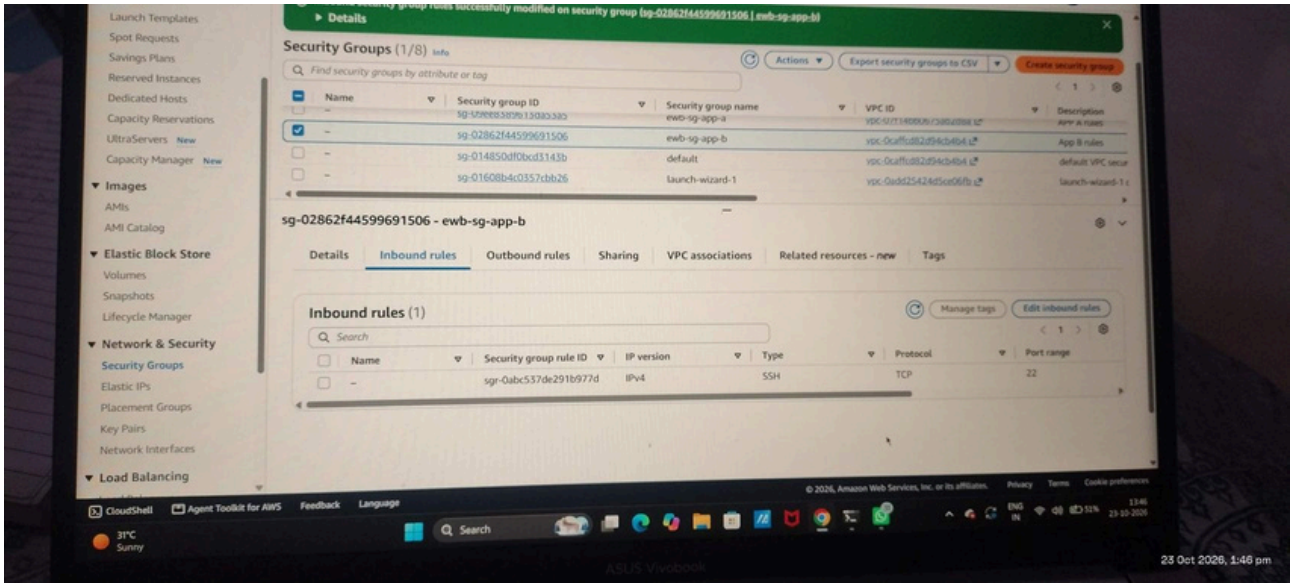


Figure 15 - ewb-sg-app-b (sg-02862f44599691506): ICMP rule removed, Inbound rules (1) = SSH TCP 22 only.

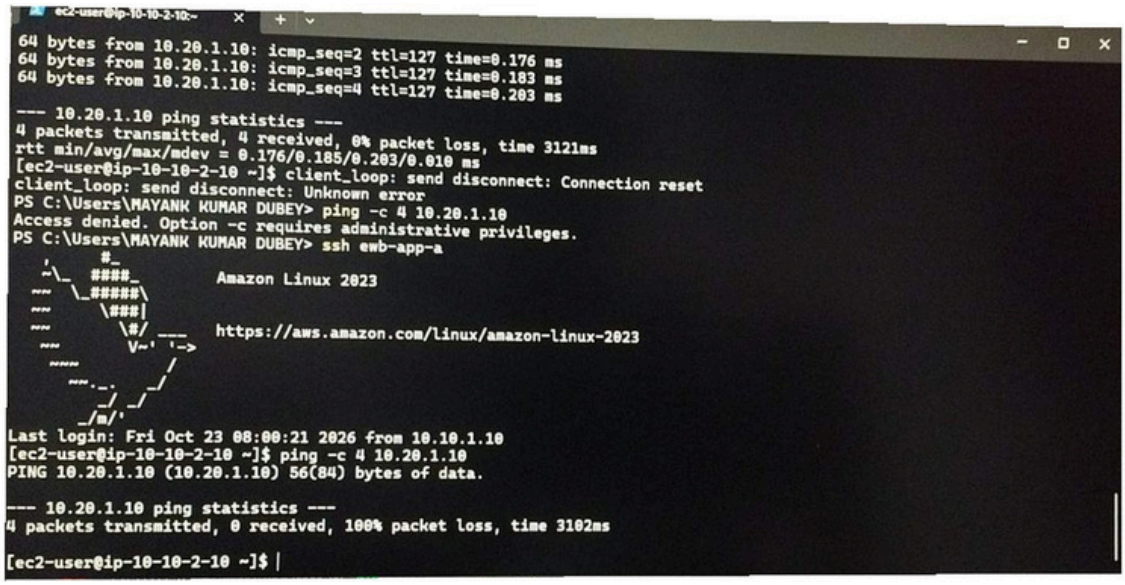


Figure 16 - Host: ewb-app-a. ping -c 4 10.20.1.10 with the ICMP rule removed: 0 received, 100% packet loss.

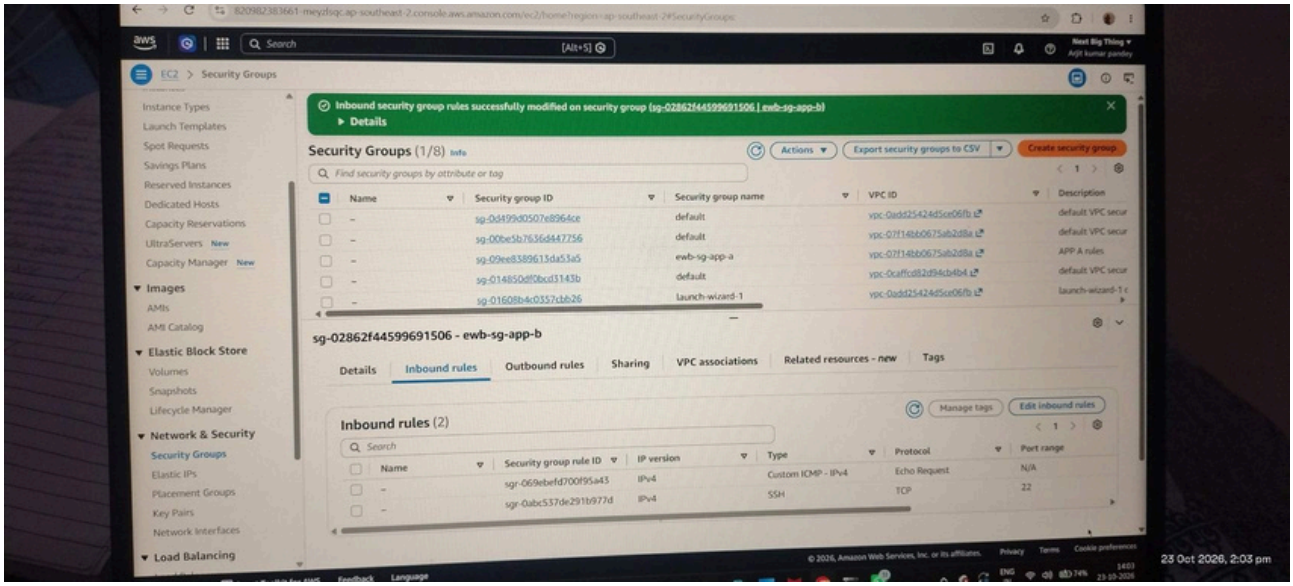


Figure 17 - ICMP Echo Request rule restored on ewb-sg-app-b: Inbound rules (2).

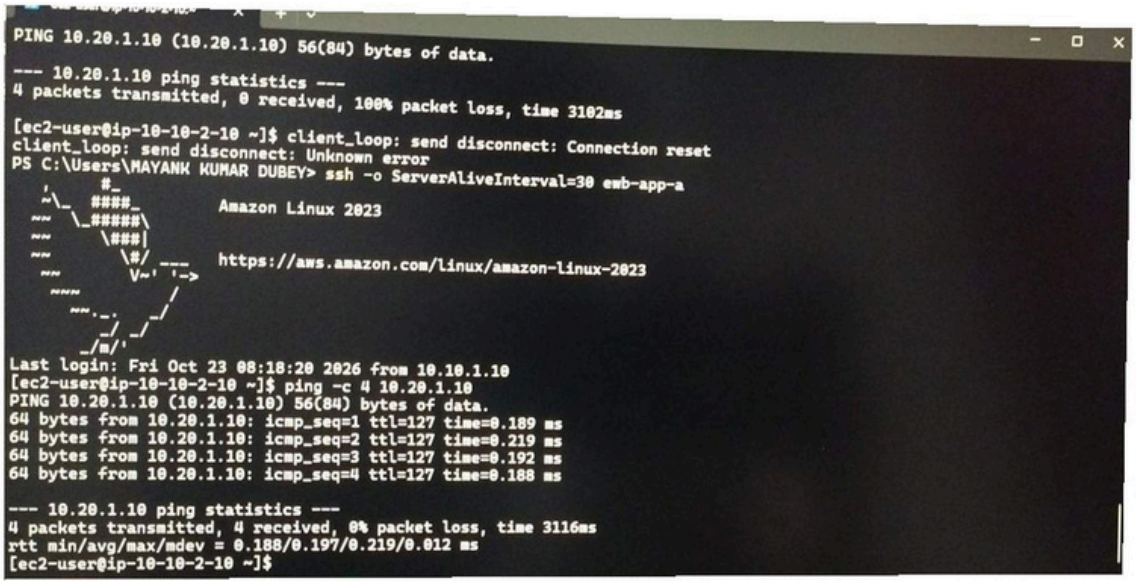


Figure 18 - Host: ewb-app-a. After restoring the rule: 4 transmitted, 4 received, 0% packet loss.

5. Forward path, return path and what I learned

Forward path

A ping from App-A (10.10.2.10) to App-B (10.20.1.10) leaves the private subnet. ewb-rt-a-private matches 10.20.0.0/16 and sends it to the peering connection pcx-0b40275e45e3598c6. In VPC B the packet reaches ewb-b-private, and ewb-sg-app-b allows ICMP Echo Request from 10.10.2.10/32.

Return path

The reply goes from 10.20.1.10 to 10.10.2.10. ewb-rt-b-private matches 10.10.0.0/16 and sends it back over the same peering connection. Security groups are stateful, so the reply is allowed automatically. Without this return route the ping would still fail, because a one-way route is not enough.

Why the SSH source on App-B is the bastion private IP

With ProxyJump the bastion opens a second SSH connection to App-B, so App-B sees 10.10.1.10 as the source. The public route table also needs 10.20.0.0/16 -> pcx so that the bastion can reach VPC B.

Other answers

Active peering does not guarantee a ping: route tables and security groups must also allow it (Figures 11-18 prove this). VPC B cannot use VPC A's internet gateway over peering, because peering is non-transitive and does not provide internet access.

Troubleshooting note: bastion SSH timeout

At first ssh ewb-bastion timed out even though my SG rule contained the IP shown by checkip.amazonaws.com. My network reaches IPv4 hosts through carrier NAT, so the bastion saw a different source address. I temporarily widened the rule to find the address, read it on the bastion with journalctl -u sshd (it was 152.59.62.39), and then narrowed the rule to 152.59.62.39/32. The rule is never opened to 0.0.0.0/0.

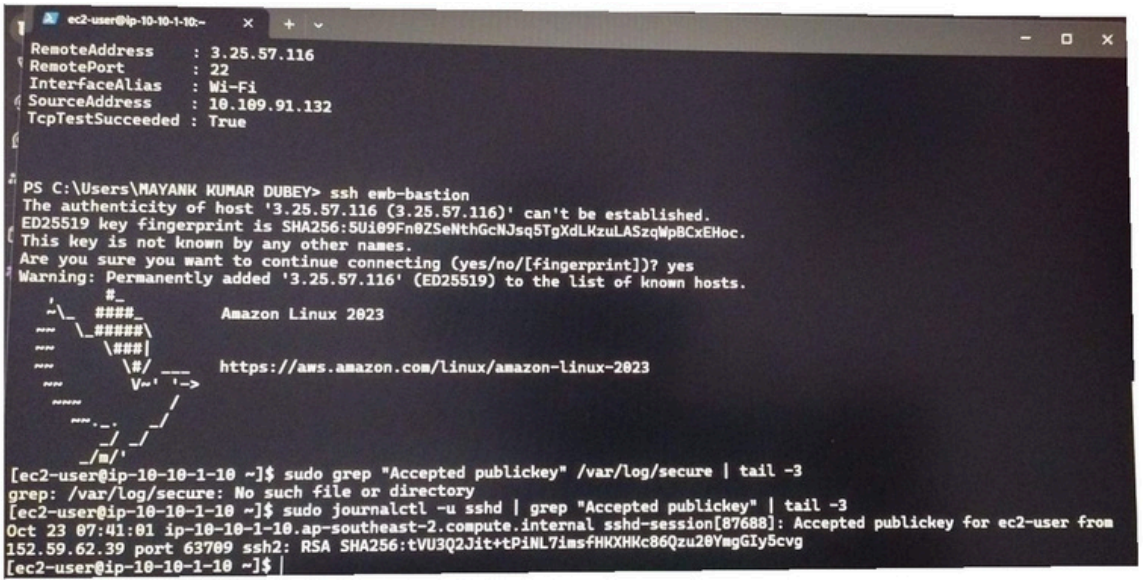


Figure 19 - Host: ewb-bastion. journalctl shows the source address seen by the bastion: 152.59.62.39.

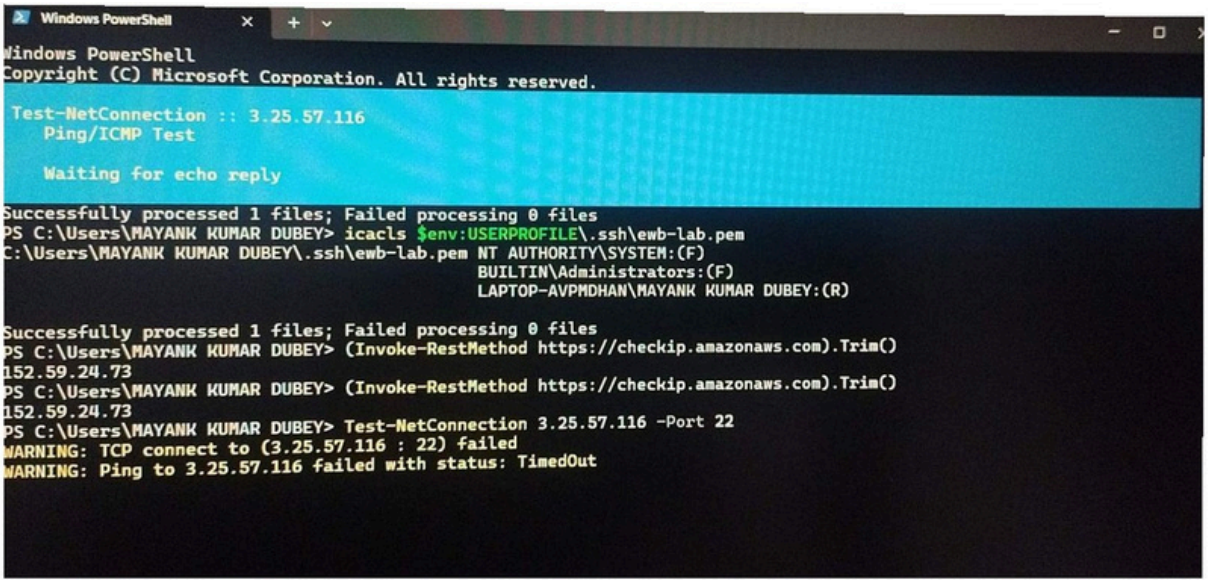


Figure 20 - Before the fix: Test-NetConnection to the bastion on port 22 timed out (TcpTestSucceeded False).

What I learned

1) A VPC peering connection needs routes on both sides. 2) Security groups on the destination must allow the peer's private IP. 3) The address AWS sees for my laptop can differ from the address shown by a lookup website. 4) Testing a failure first, and then fixing it, proves which component makes the connection work.